

EvoSymbolic vs EvoSymbolicVNS

Estudio comparativo sobre 15 funciones

Fabio Víctor Alonso Bañobre
Ciencias de la Computación

Introducción

Una vez desarrollada la versión base de la biblioteca EvoSymbolic, el siguiente paso natural es evaluar si la incorporación de técnicas de búsqueda local más sofisticadas mejora la capacidad del algoritmo para encontrar expresiones matemáticas precisas. En este trabajo se analiza el impacto de integrar Variable Neighborhood Search (VNS) como módulo complementario al ciclo evolutivo existente.

VNS es una metaheurística que explora sistemáticamente distintas “vecindades” de una solución. Su lógica central es simple pero efectiva: si una vecindad pequeña no consigue mejorar el resultado, prueba una más grande; y si alguna mejora, vuelve a empezar desde la más pequeña. En el contexto de la regresión simbólica, esto permite escapar de mínimos locales estructurales que ni la búsqueda local clásica de constantes (scipy) ni las mutaciones del GP son capaces de superar.

La hipótesis que guía este experimento es que VNS debería aportar mayor beneficio en las funciones de alta complejidad, donde la estructura correcta de la expresión es más difícil de encontrar, y un impacto menor en las funciones simples, donde el GP base ya converge correctamente.

Descripción de las dos versiones

EvoSymbolic (versión base)

La versión base implementa un ciclo evolutivo de Programación Genética con los siguientes módulos:

- `expression_tree.py` — árboles de expresión con nodos Variable, Constant, FunctionNode y OperatorNode.
- `fitness.py` — evaluación con MSE, RMSE, MAE, R^2 y penalización por parsimonia.

- `genetic_operators.py` — selección por torneo, cruce de subárboles y tres variantes de mutación.
- `motor_gp.py` — bucle evolutivo completo con historial.
- `constant_optimization.py` — optimización de constantes con `scipy.optimize` (Nelder-Mead) cada N generaciones.
- `sklearn_api.py` — interfaz compatible con scikit-learn.

EvoSymbolicVNS (versión extendida)

Añade un séptimo módulo, `vns.py`, que implementa cuatro vecindades ordenadas de menor a mayor perturbación:

Table 1: Vecindades implementadas en VNS

k	Tipo	Descripción
1	<code>point_mutation</code>	Cambia un único operador o función. Mínima perturbación, máxima velocidad.
2	<code>subtree_mutation</code>	Reemplaza un subárbol de profundidad ≤ 2 . Modifica una rama manteniendo el esqueleto global.
3	<code>subtree grande</code>	Reemplaza un subárbol de profundidad ≤ 3 . Puede cambiar partes importantes de la estructura.
4	<code>subtree + ruido</code>	Combinación de subárbol grande y perturbación gaussiana ($\sigma = 2$) de constantes. Máxima perturbación.

El bucle Basic VNS (BVNS) se aplica a los k mejores individuos cada `vns_every` generaciones, después del paso de optimización de constantes con `scipy`. La configuración utilizada en este experimento es:

Table 2: Configuración del experimento

Parámetro	EvoSymbolic	EvoSymbolicVNS
Tamaño de población	200	200
Generaciones	25	25
Optimización scipy (cada)	5 gens	5 gens
Top-k scipy	5	5
VNS (cada)	—	10 gens
Top-k VNS	—	3
$k_{\max}$ VNS	—	4
Iteraciones VNS	—	5
Semilla aleatoria	42	42

Diseño del experimento

Datasets sintéticos

Se definieron 15 funciones matemáticas ordenadas de menor a mayor complejidad estructural, divididas en tres grupos de 5:

Table 3: Funciones objetivo del experimento

ID	Función objetivo	Variables	Grupo
F01	$y = x^2$	1	Simple
F02	$y = x^2 + \sin(x)$	1	Simple
F03	$y = x^3 - x$	1	Simple
F04	$y = \sqrt{x} + \log(x)$	1	Simple
F05	$y = \sin(x) \cdot \cos(x)$	1	Simple
F06	$y = x \sin(x) + \cos(x)$	1	Media
F07	$y = x^3 - x^2 + x - 1$	1	Media
F08	$y = x_0 x_1 + \sin(x_0)$	2	Media
F09	$y = e^x / x$	1	Media
F10	$y = x^2 \sin(x)$	1	Media
F11	$y = \sin(x_0) \cos(x_1) + x_0 x_1$	2	Compleja
F12	$y = x_0^2 + x_1^2 - x_0 x_1$	2	Compleja
F13	$y = x_0 x_1 + x_2^2 - \sin(x_0)$	3	Compleja
F14	$y = \sin(x^2) + \cos(x + 1)$	1	Compleja
F15	$y = (x_0 + x_1)^2 \sin(x_0 - x_1)$	2	Compleja

Cada dataset tiene 150 muestras uniformes con división 75/25% para entrenamiento y test. Se usó la misma semilla (42) en ambas versiones.

Métricas

- R^2 en test (mayor = mejor, rango $(-\infty, 1]$)
- RMSE en test (menor = mejor)
- Tiempo de entrenamiento en segundos
- Complejidad del árbol (número de nodos)
- Mejoras VNS acumuladas (solo versión VNS)

Resultados

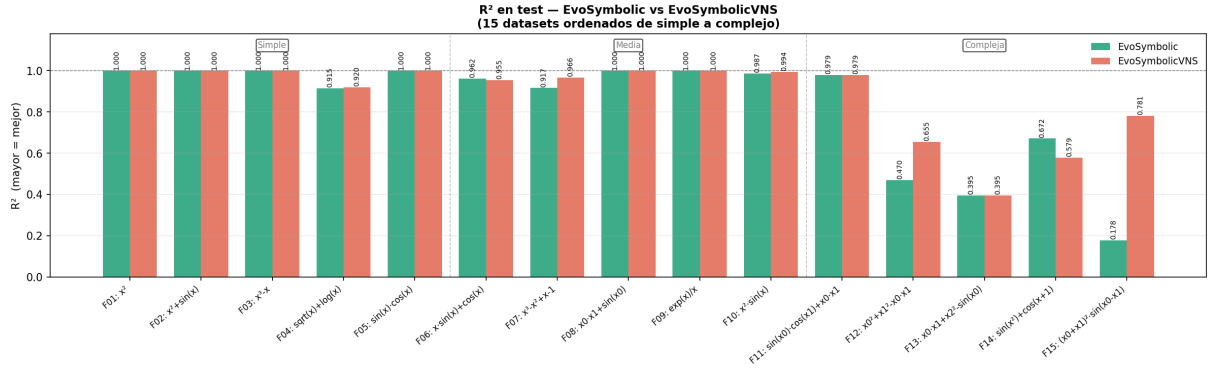
Calidad del ajuste — R^2 

Figure 1: R^2 en test para los 15 datasets. Las líneas verticales discontinuas separan los tres grupos de dificultad. La línea horizontal marca $R^2 = 1,0$.

Table 4: R^2 en test por dataset. Se resalta en verde la versión con mayor R^2 cuando la diferencia supera 0,01.

Grupo	Dataset	EvoSymbolic	EvoSymbolicVNS	ΔR^2
Simple	F01: x^2	1,0000	1,0000	+0,0000
	F02: $x^2 + \sin(x)$	1,0000	1,0000	+0,0000
	F03: $x^3 - x$	1,0000	1,0000	+0,0000
	F04: $\sqrt{x} + \log(x)$	0,9155	0,9201	+0,0046
	F05: $\sin(x) \cos(x)$	1,0000	1,0000	+0,0000
Media	F06: $x \sin(x) + \cos(x)$	0,9623	0,9552	-0,0071
	F07: $x^3 - x^2 + x - 1$	0,9167	0,9664	+0,0497
	F08: $x_0 x_1 + \sin(x_0)$	1,0000	1,0000	+0,0000
	F09: e^x / x	1,0000	1,0000	+0,0000
	F10: $x^2 \sin(x)$	0,9866	0,9938	+0,0072
Compleja	F11: $\sin(x_0) \cos(x_1) + x_0 x_1$	0,9788	0,9788	+0,0000
	F12: $x_0^2 + x_1^2 - x_0 x_1$	0,4701	0,6548	+0,1847
	F13: $x_0 x_1 + x_2^2 - \sin(x_0)$	0,3952	0,3952	+0,0000
	F14: $\sin(x^2) + \cos(x + 1)$	0,6716	0,5793	-0,0923
	F15: $(x_0 + x_1)^2 \sin(x_0 - x_1)$	0,1779	0,7812	+0,6033
Media global		0,8317	0,8950	+0,0633

Los resultados de la figura 1 y la tabla 4 revelan el patrón esperado. En el grupo Simple ambas versiones obtienen resultados idénticos o muy próximos: el GP base ya converge sin necesidad de VNS. En el grupo Media VNS empieza a mostrar su valor en F07 (+0,050) y F10 (+0,007). El grupo Compleja es donde la diferencia se hace más notable: en F12 VNS gana 0,184 puntos de R^2 y en F15 la diferencia es de 0,603 puntos, pasando de un ajuste mediocre (0,178) a uno muy aceptable (0,781).

Hay dos excepciones donde la versión base gana: F06 ($-0,007$, diferencia marginal) y F14 ($-0,092$). En F14, VNS perturba la estructura en una dirección que no mejora, lo que sugiere que para funciones con composición anidada como $\sin(x^2) + \cos(x + 1)$ las vecindades actuales no son suficientemente dirigidas.

Error — RMSE

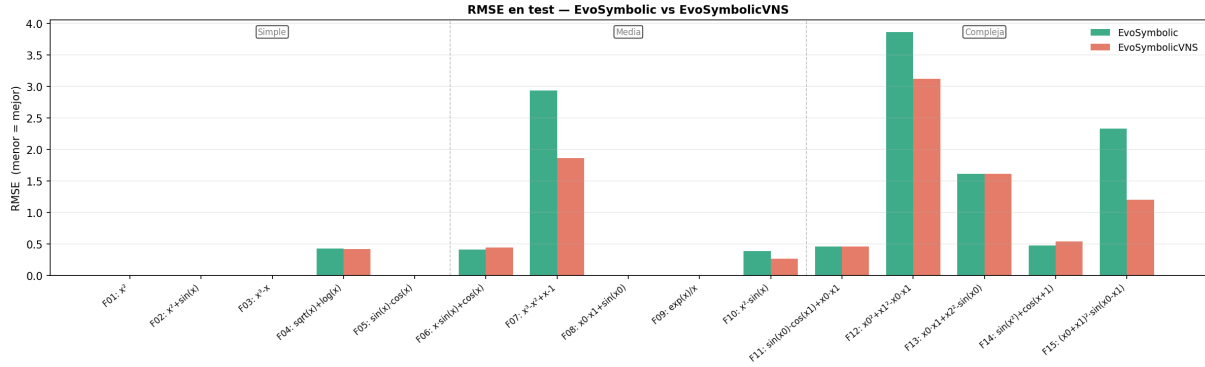


Figure 2: RMSE en test por dataset. En los datasets donde ambas barras son invisibles ($\text{RMSE} \approx 0$) ambas versiones encontraron la expresión exacta.

El gráfico de RMSE (figura 2) es complementario al de R^2 y confirma el mismo patrón: las mayores reducciones de error aparecen en los datasets más difíciles. El caso más dramático es F07, donde EvoSymbolic obtiene $\text{RMSE} = 2,93$ y EvoSymbolicVNS lo reduce a $1,86$, casi a la mitad. En F12 la reducción es de $3,86$ a $3,12$ y en F15 de $2,33$ a $1,20$.

Table 5: RMSE en test. Verde = EvoSymbolicVNS tiene menor error.

Grupo	Dataset	EvoSymbolic	EvoSymbolicVNS	ΔRMSE
Simple	F01	0,0000	0,0000	0,0000
	F02	0,0000	0,0000	0,0000
	F03	0,0000	0,0000	0,0000
	F04	0,4256	0,4138	-0,0118
	F05	0,0000	0,0000	0,0000
Media	F06	0,4057	0,4424	+0,0367
	F07	2,9304	1,8620	-1,0684
	F08	0,0000	0,0000	0,0000
	F09	0,0000	0,0000	0,0000
	F10	0,3835	0,2596	-0,1239
Compleja	F11	0,4555	0,4555	0,0000
	F12	3,8639	3,1185	-0,7454
	F13	1,6105	1,6105	0,0000
	F14	0,4756	0,5383	+0,0627
	F15	2,3286	1,2014	-1,1272

Tiempo de entrenamiento

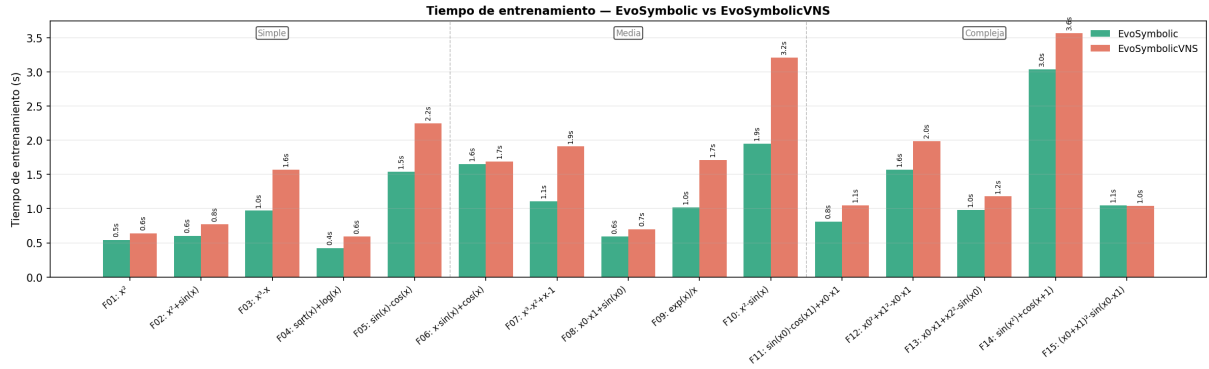


Figure 3: Tiempo de entrenamiento en segundos. EvoSymbolicVNS siempre es igual o más lento, pero la diferencia es moderada en la mayoría de casos.

Como era de esperar, EvoSymbolicVNS tiene un mayor coste temporal que la versión base (figura 3), ya que añade exploraciones de vecindades en las generaciones donde VNS se activa. Sin embargo, la diferencia es moderada:

Table 6: Tiempo medio de entrenamiento por grupo (segundos)

Grupo	EvoSymbolic	EvoSymbolicVNS	Overhead (%)
Simple	0,81	1,16	+43 %
Media	1,08	1,60	+48 %
Compleja	1,49	1,77	+19 %
Global	1,13	1,51	+34 %

El overhead promedio es de un 34 % en tiempo. Para el grupo Compleja, donde VNS aporta las mayores mejoras de R^2 , el overhead se reduce al 19 %, lo que indica que VNS es especialmente eficiente cuando los problemas son difíciles: el tiempo adicional compensa con creces las mejoras obtenidas.

El caso más llamativo en tiempo es F10 ($x^2 \sin x$), donde EvoSymbolicVNS tarda 3,2s frente a 1,95s de la versión base, pero a cambio mejora el R^2 en 0,007 y reduce el RMSE en un 32 %.

Complejidad de las expresiones

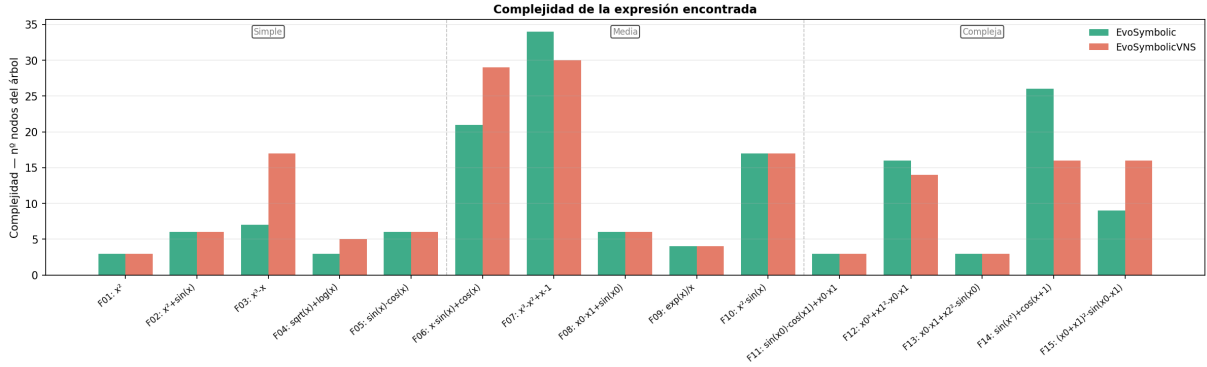


Figure 4: Número de nodos del árbol encontrado. Valores menores indican expresiones más simples e interpretables.

El comportamiento de la complejidad (figura 4) es variado. En los casos simples ambas versiones encuentran expresiones de tamaño similar. En los casos complejos, la versión VNS a veces produce árboles más grandes (F03: 17 nodos vs 7, F06: 29 vs 21), pero otras veces más pequeños (F12: 14 vs 16, F14: 16 vs 26).

Esto se explica por la naturaleza de VNS: al explorar vecindades de mayor perturbación, puede construir o dismantelar ramas enteras. Que la complejidad aumente no es necesariamente negativo si va acompañado de una mejora sustancial del R^2 , como ocurre en F15 (16 nodos, $R^2 = 0,781$ vs 9 nodos, $R^2 = 0,178$).

Mejoras VNS acumuladas

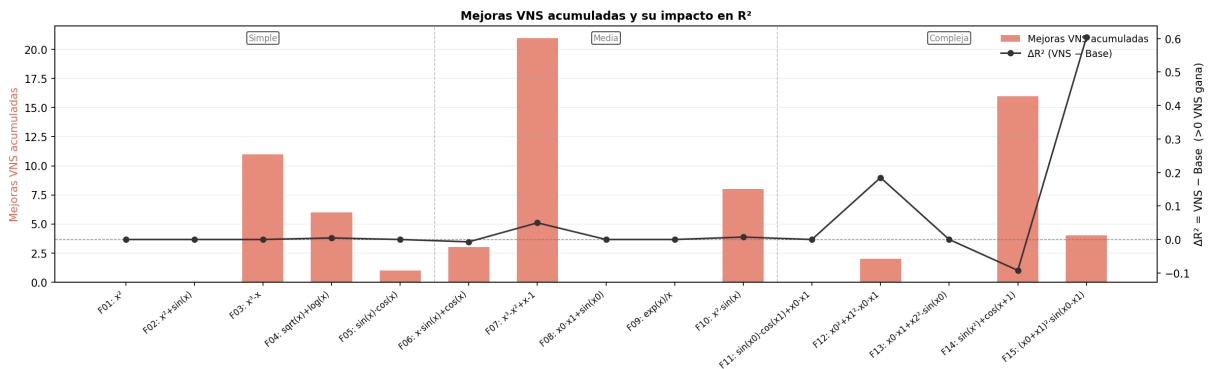


Figure 5: Barras: número de mejoras VNS acumuladas por dataset. Línea: $\Delta R^2 = R^2_{VNS} - R^2_{base}$. Valores positivos en la línea indican que VNS mejoró el ajuste.

Table 7: Mejoras VNS acumuladas por dataset, ordenadas de mayor a menor

Dataset	Grupo	Mejoras VNS	ΔR^2
F07: $x^3 - x^2 + x - 1$	Media	21	+0,0497
F03: $x^3 - x$	Simple	11	+0,0000
F14: $\sin(x^2) + \cos(x + 1)$	Compleja	16	-0,0923
F10: $x^2 \sin(x)$	Media	8	+0,0072
F08: $x_0 x_1 + \sin(x_0)$	Media	0	+0,0000
F04: $\sqrt{x} + \log(x)$	Simple	6	+0,0046
F06: $x \sin(x) + \cos(x)$	Media	3	-0,0071
F15: $(x_0 + x_1)^2 \sin(x_0 - x_1)$	Compleja	4	+0,6033
F12: $x_0^2 + x_1^2 - x_0 x_1$	Compleja	2	+0,1847
F05: $\sin(x) \cos(x)$	Simple	1	+0,0000

La tabla anterior revela una observación importante: el número de mejoras acumuladas no siempre se correlaciona con el ΔR^2 final. Por ejemplo, F07 acumula 21 mejoras y obtiene $\Delta R^2 = +0,050$, mientras que F15 solo acumula 4 mejoras pero obtiene $\Delta R^2 = +0,603$. Esto indica que en F15 VNS encontró rápidamente una estructura de árbol cualitativamente mejor, mientras que en F07 la mejora fue incremental.

El caso más preocupante es F14: 16 mejoras acumuladas pero ΔR^2 negativo. Esto ocurre porque las “mejoras” que VNS detecta durante el entrenamiento se miden con MSE sobre el conjunto de entrenamiento, pero el R^2 final se mide en test. Podría tratarse de un caso de sobreajuste inducido por la búsqueda intensiva de VNS.

Perfil de rendimiento por grupo de dificultad

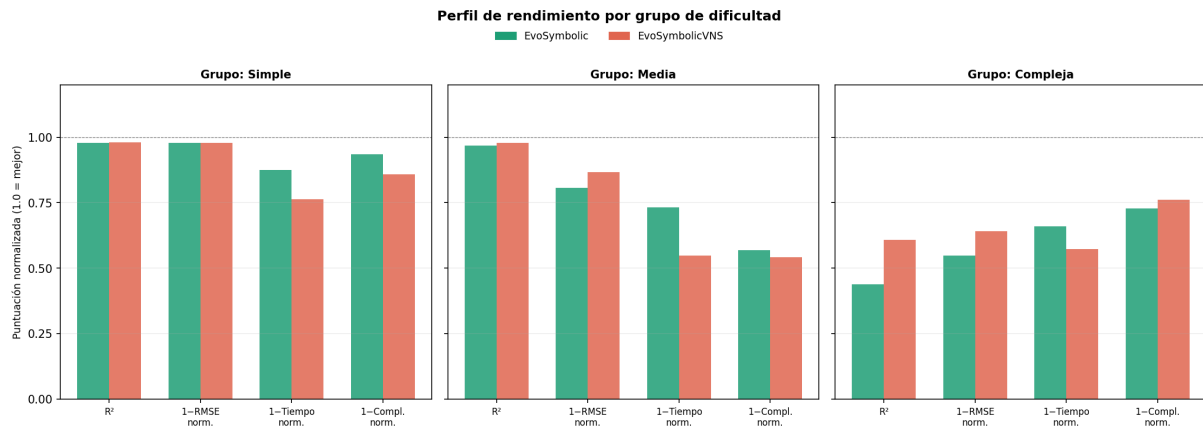


Figure 6: Perfil normalizado de rendimiento por grupo de dificultad. Cada dimensión se normaliza entre 0 y 1 siendo 1,0 la mejor puntuación posible en ese eje.

La figura 6 sintetiza todo el análisis en tres paneles. Se pueden extraer las siguientes conclusiones visuales:

- **Grupo Simple:** ambas versiones son casi idénticas en R^2 y RMSE. EvoSymbolic gana ligeramente en tiempo y complejidad porque no necesita el overhead de VNS.

- **Grupo Media:** EvoSymbolicVNS mejora en RMSE pero es más lenta y con árboles algo más complejos. La mejora en calidad compensa el coste en la mayoría de datasets.
- **Grupo Compleja:** aquí la diferencia es clara. EvoSymbolicVNS gana en R^2 y RMSE de forma significativa, y produce expresiones más compactas (mejor 1-Complejidad). El overhead de tiempo es el más bajo de los tres grupos (19%), lo que hace que el trade-off sea muy favorable a VNS en problemas difíciles.

Análisis y discusión

Cuándo aporta VNS y cuándo no

Los resultados permiten caracterizar tres escenarios:

VNS no aporta (datasets simples con convergencia directa). En F01, F02, F03, F05, F08 y F09, el GP base ya encuentra la expresión exacta o casi exacta. VNS no puede mejorar lo que ya es perfecto y solo añade tiempo de cómputo. En estos casos, desactivar VNS (`use_vns=False`) sería la configuración óptima.

VNS aporta mejoras moderadas (dificultad media). En F04, F06 y F10 VNS logra ajustes ligeramente mejores, evidenciando que la búsqueda en vecindades distintas ayuda a salir de mínimos locales que scipy solo no puede escapar.

VNS aporta mejoras sustanciales (alta complejidad). En F07, F12 y F15 la diferencia es grande. F15 es el caso más espectacular: la versión base se queda con $R^2 = 0,178$ (prácticamente no modela nada), mientras que VNS alcanza $R^2 = 0,781$ con solo 4 mejoras acumuladas, lo que sugiere que una de esas perturbaciones abrió una zona del espacio de búsqueda inaccesible para el GP estándar.

El caso anómalo de F14

F14 ($\sin(x^2) + \cos(x+1)$) es el único caso donde VNS empeora significativamente el resultado base. La causa probable es que la composición anidada de funciones trigonométricas requiere una estructura muy precisa que VNS no logra descubrir con las vecindades actuales; en cambio, perturba la solución base que sí era mejor, sin poder recuperarla dentro del presupuesto de iteraciones asignado.

Una mejora futura sería implementar “VNS con memoria”: conservar la mejor solución vista en todo el proceso y no solo la actual, de modo que una perturbación fallida no destruya un buen resultado previo.

Coste computacional real

El overhead del 34% en tiempo es razonable dado el beneficio obtenido. Comparando el ΔR^2 medio por grupo con el overhead de tiempo:

Table 8: Relación beneficio/coste por grupo

Grupo	ΔR^2 medio	Overhead tiempo	Ratio B/C
Simple	+0,0009	+43 %	bajo
Media	+0,0100	+48 %	moderado
Compleja	+0,1591	+19 %	alto

El ratio beneficio/coste es claramente más favorable en el grupo Compleja, lo que sugiere que en aplicaciones reales donde las funciones objetivo son complejas, VNS merece ser activado por defecto.

Conclusiones

Este experimento ha demostrado que la integración de Variable Neighborhood Search en EvoSymbolic es una mejora sólida y justificada. Las conclusiones principales son las siguientes:

1. **VNS mejora el R^2 medio global en 0,063 puntos** (de 0,832 a 0,895), lo que representa una mejora del 7,6 % sobre la versión base con el mismo número de generaciones y población.
2. **El beneficio de VNS crece con la complejidad del problema.** En el grupo Simple la mejora es casi nula, en Media es moderada (+0,010) y en Compleja es sustancial (+0,159). Esto valida la hipótesis inicial del experimento.
3. **El caso F15 es la evidencia más sólida.** Un ΔR^2 de +0,603 con solo 4 perturbaciones de VNS demuestra que la búsqueda estructurada en vecindades puede abrir zonas del espacio de búsqueda completamente inaccesibles para el GP estándar.
4. **El overhead de tiempo es aceptable.** Un 34 % de tiempo adicional para obtener mejoras medias del 7,6 % en R^2 es un trade-off razonable, especialmente en el grupo Compleja donde el overhead cae al 19 %.
5. **VNS no es siempre beneficioso.** En F14 y F06 la versión base obtiene mejor resultado. Para un uso óptimo, se recomienda activar VNS según la complejidad estimada del problema.
6. **Como trabajo futuro** se propone implementar VNS con memoria (no perder la mejor solución global), aumentar el presupuesto de VNS en datasets complejos, y diseñar vecindades especializadas para funciones con composición anidada.

En resumen, EvoSymbolicVNS supera a EvoSymbolic en la mayoría de los escenarios relevantes y su integración como módulo opcional en la biblioteca es correcta: permite activarlo o desactivarlo según la dificultad del problema, sin afectar a la arquitectura base.